TDDE31-Report2

Ahmad Soltani & Fangyun Yang

April 2026

Assignment 1

Maximum Temperatures (Descending)

Year	StationID	maxValue (°C)	Year	StationID	maxValue (°C)
1975	86200	36.1	2013	98210	31.6
1992	63600	35.4	2009	81370	31.5
1994	117160	34.7	2012	54990	31.3
2010	75250	34.4	1964	76430	31.2
2014	96560	34.4	1971	65130	31.2
1989	63050	33.9	1972	137100	31.2
1982	94050	33.8	1976	75040	31.1
1968	137100	33.7	1961	76000	31.0
1966	151640	33.5	1963	62410	31.0
1983	98210	33.3	1958	117160	30.8
2002	78290	33.3	1978	71470	30.8
1970	103080	33.2	1995	76420	30.8
1986	76470	33.2	1996	96140	30.8
1956	145340	33.0	1974	167710	30.6
2000	62400	33.0	1954	53650	30.5
1959	65160	32.8	1952	106100	30.4
1991	137040	32.7	1980	161790	30.4
2006	75240	32.7	1990	63340	30.2
1988	102540	32.6	2004	117160	30.2
2011	172770	32.5	1957	75040	29.8
1999	98210	32.4	1985	85450	29.8
1953	65160	32.2	1981	65160	29.7
1955	97260	32.2	1993	132030	29.7
1973	71470	32.2	1987	96310	29.6
2003	136420	32.2	1967	75040	29.5
2007	86420	32.2	1984	105370	29.5
2008	102390	32.2	1950	75040	29.4
2005	107140	32.1	1960	173810	29.4
1969	71470	32.0	1998	63600	29.2
1979	63600	32.0	1951	75040	28.5
2001	62400	31.9	1965	116500	28.5
1977	94180	31.8	1962	86200	27.4
1997	74180	31.8			

Minimum Temperatures (Descending)

Year	StationID	minValue (°C)	Year	StationID	minValue (°C)
1990	147270	-35.0	1969	181900	-41.5
1952	192830	-35.5	2003	179960	-41.5
1974	179950	-35.6	1996	155790	-41.7
1954	113410	-36.0	2010	191910	-41.7
1992	179960	-36.1	1950	155910	-42.0
1975	157860	-37.0	1951	155910	-42.0
1972	167860	-37.5	1962	181900	-42.0
1995	182910	-37.6	1968	179950	-42.0
2000	169860	-37.6	2011	179960	-42.0
1957	159970	-37.8	1976	192830	-42.2
1983	191900	-38.2	1982	113410	-42.2
1989	166870	-38.2	2002	169860	-42.2
1953	183760	-38.4	1977	179950	-42.5
2009	179960	-38.5	2014	192840	-42.5
1993	191900	-39.0	1998	169860	-42.7
1984	123480	-39.2	2012	191910	-42.7
1973	166870	-39.3	1958	159970	-43.0
1991	179960	-39.3	1985	159970	-43.4
2008	179960	-39.3	1959	159970	-43.6
2005	155790	-39.4	1965	189780	-44.0
1961	181900	-39.5	1979	112170	-44.0
1964	166810	-39.5	1981	166870	-44.0
1970	179950	-39.6	2001	112530	-44.0
2004	166940	-39.7	1986	167860	-44.2
1988	170790	-39.9	1971	166870	-44.3
1960	155910	-40.0	1956	160790	-45.0
1997	179960	-40.2	1980	191900	-45.0
1994	179960	-40.5	1967	166870	-45.4
2006	169860	-40.6	1987	123480	-47.3
2007	169860	-40.7	1978	155940	-47.7
2013	179960	-40.7	1999	192830	-49.0
1963	181900	-41.0	1966	179950	-49.4
1955	160790	-41.2			

Assinment 2

Part 1: Monthly Count of Temperature Readings above 10°C (1950-2014)

Top Readings			Bottom Readings		
Year	**Month**	**Count**	**Year**	**Month**	**Count**
2014	07	147681	1970	03	1
2011	07	146656	1979	01	1
2010	07	143419	1950	12	1
2012	07	137477	1960	01	1
2013	07	133657	1993	01	1
2009	07	133008	1962	02	1
2011	08	132734	1960	02	1
2009	08	128349	1975	12	1
2013	08	128235	1954	02	1
2003	07	128133	1951	02	1
2002	07	127956	2001	03	1
2006	08	127622	1952	11	1
2008	07	126973	1958	02	1
2002	08	126073	1984	03	1
2005	07	125294	1995	02	1
2011	06	125193	1988	11	1
2012	08	125037	1958	01	1
2006	07	124794	1984	01	1
2010	08	124417	1962	03	1
2014	08	124045	1952	03	1

Part 2: Distinct Stations Monthly Count above 10°C (1950-2014)

Top Readings			Bottom Readings		
Year	**Month**	**Count**	**Year**	**Month**	**Count**
1972	10	378	1960	02	1
1973	06	377	2005	12	1
1973	05	377	2001	03	1
1972	08	376	1952	03	1
1973	09	376	1993	11	1
1972	05	375	2014	12	1
1972	09	375	1970	03	1
1972	06	375	1975	12	1
1971	08	375	1950	12	1
1971	06	374	1952	11	1
1972	07	374	1995	02	1
1971	09	374	1987	12	1
1973	08	373	1962	03	1
1971	05	373	1988	11	1
1974	06	372	1962	02	1
1974	08	372	1958	01	1
1974	09	370	1960	01	1
1973	07	370	1991	01	1
1971	07	370	1958	02	1
1974	05	370	1984	01	1

Assinment 3

Average Monthly Temperature by Station in Sweden (1960-2014)

Top 20 Highest Temperatures				Bottom 20 Lowest Temperatures			
Year	**Month**	**Station**	**Avg Temp (°C)**	**Year**	**Month**	**Station**	**Avg Temp (°C)**
2014	07	96000	26.300	1985	02	160890	-23.599
1994	07	65450	23.655	1985	02	167980	-23.605
1994	07	95160	23.505	1985	02	182930	-23.627
1994	07	75120	23.269	1966	02	191900	-23.632
1994	07	105260	23.144	1985	02	166810	-23.636
1994	07	85280	23.109	1985	02	179950	-23.642
1983	08	54550	23.000	1985	02	183980	-23.702
1975	08	54550	22.963	1993	12	166900	-23.800
1994	07	96550	22.958	1985	02	159970	-23.962
1994	07	96000	22.931	1966	02	189780	-23.978
1994	07	106070	22.823	1966	02	192830	-24.025
1972	07	173960	22.777	1966	02	181900	-24.093
1994	07	54300	22.760	1966	02	166870	-24.229
1994	07	85210	22.756	1985	02	183760	-24.481
2006	07	65450	22.741	1966	02	159970	-24.692
2006	07	75120	22.730	1966	02	167860	-24.728
1994	07	103080	22.709	1985	02	166870	-24.767
1994	07	92100	22.699	1985	02	169880	-25.408
1994	07	94180	22.682	1985	02	192830	-26.191
1994	07	83230	22.577	1985	02	181900	-26.674

Assignment 4

Output: Stations with maximum temperatures between 25–30 °C and precipitation between 50–200 mm

StationID	Max temp (°C)	Max prec (mm)	StationID	Max temp (°C)	Max prec (mm)
189720	28.1	56.2	123060	28.0	58.8
188850	27.9	70.0	114410	29.1	54.3
182910	29.2	54.5	114140	30.0	50.5
179960	27.9	54.1	103420	29.7	62.3
166910	27.1	65.5	94190	30.0	64.9
162790	28.2	55.5	87140	29.5	52.9
158740	29.2	60.8	84340	30.0	61.8
157870	29.9	60.9	82360	29.9	84.3
144310	28.4	53.5	81050	29.0	54.1
139260	29.4	86.2	78280	28.7	51.0
139120	27.9	93.9	71190	28.3	79.2
135460	29.2	61.4	68560	28.5	52.0
133500	27.2	50.1	66110	26.3	72.6
124300	27.2	93.9	52240	29.0	78.9

Answer:

In this exercise, no records were initially found when filtering precipitation values between 100 and 200 mm. This absence of results may be due to logical errors in the code or a misunderstanding of the assignment requirements. After adjusting the precipitation range to between 50 and 200 mm, valid records were obtained and are presented in the table.

The records include station numbers, their corresponding maximum temperatures (restricted to the range of 25–30 °C), and the corresponding maximum daily precipitation values. The results shown in the table are sorted by maximum temperature in descending order.

Assinment 5

Output: Average Monthly Precipitation (1993–2016)

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
1993	04	0.0	2004	08	75.5
1993	05	21.1	2004	09	37.2
1993	06	56.5	2004	10	78.2
1993	07	95.4	2004	11	54.1
1993	08	80.7	2004	12	24.2
1993	09	40.6	2005	01	18.1
1993	10	43.2	2005	02	33.5
1993	11	42.8	2005	03	23.5
1993	12	37.1	2005	04	11.6
1994	01	22.1	2005	05	55.4
1994	02	22.5	2005	06	68.0
1994	03	37.6	2005	07	104.3
1994	04	23.1	2005	08	77.0
1994	05	25.1	2005	09	14.0
1994	06	45.1	2005	10	38.1
1994	07	0.0	2005	11	32.6
1994	08	58.8	2005	12	56.6
1994	09	94.6	2006	01	17.7
1994	10	33.2	2006	02	34.8
1994	11	15.7	2006	03	27.9
1994	12	50.9	2006	04	44.4
1995	01	26.0	2006	05	52.3
1995	02	31.8	2006	06	31.1
1995	03	34.4	2006	07	29.0
1995	04	61.7	2006	08	148.1
1995	05	26.0	2006	09	19.3
1995	06	97.2	2006	10	118.2
1995	07	43.6	2006	11	71.7

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
1995	08	16.0	2006	12	29.7
1995	09	134.6	2007	01	68.6
1995	10	20.9	2007	02	33.1
1995	11	31.6	2007	03	40.5
1995	12	5.1	2007	04	21.2
1996	01	10.4	2007	05	40.5
1996	02	15.7	2007	06	108.9
1996	03	10.0	2007	07	96.0
1996	04	8.1	2007	08	54.2
1996	05	63.2	2007	09	61.9
1996	06	51.7	2007	10	28.1
1996	07	84.0	2007	11	50.7
1996	08	37.7	2007	12	54.7
1996	09	57.5	2008	01	45.0
1996	10	22.4	2008	02	28.3
1996	11	67.1	2008	03	42.2
1996	12	39.6	2008	04	20.2
1997	01	5.8	2008	05	23.1
1997	02	35.4	2008	06	42.9
1997	03	9.5	2008	07	85.2
1997	04	26.0	2008	08	138.5
1997	05	60.8	2008	09	47.4
1997	06	87.0	2008	10	59.6
1997	07	42.0	2008	11	46.8
1997	08	24.6	2008	12	43.5
1997	09	43.6	2009	01	15.9
1997	10	58.2	2009	02	24.8
1997	11	64.5	2009	03	34.5
1997	12	69.6	2009	04	2.8
1998	01	44.3	2009	05	54.2
1998	02	50.0	2009	06	49.8
1998	03	33.9	2009	07	113.2
1998	04	44.4	2009	08	61.6
1998	05	38.4	2009	09	30.0
1998	06	88.9	2009	10	56.8
1998	07	85.2	2009	11	64.2
1998	08	86.5	2009	12	53.5
1998	09	56.8	2010	01	36.0
1998	10	53.4	2010	02	52.8
1998	11	29.0	2010	03	23.9
1998	12	59.4	2010	04	23.8
1999	01	61.9	2010	05	67.2
1999	02	25.1	2010	06	48.7
1999	03	42.2	2010	07	92.4
1999	04	54.6	2010	08	108.1
1999	05	27.4	2010	09	43.1
1999	06	50.3	2010	10	52.5

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
1999	07	29.1	2010	11	93.5
1999	08	54.8	2010	12	37.2
1999	09	55.5	2011	01	35.1
1999	10	18.5	2011	02	24.5
1999	11	18.4	2011	03	19.8
1999	12	66.0	2011	04	14.9
2000	01	18.6	2011	05	37.8
2000	02	24.1	2011	06	88.3
2000	03	21.7	2011	07	94.9
2000	04	36.2	2011	08	86.3
2000	05	25.3	2011	09	52.6
2000	06	62.0	2011	10	43.8
2000	07	135.9	2011	11	13.5
2000	08	54.9	2011	12	42.1
2000	09	27.5	2012	01	43.6
2000	10	110.3	2012	02	28.7
2000	11	108.1	2012	03	8.5
2000	12	63.3	2012	04	62.8
2001	01	36.4	2012	05	23.0
2001	02	36.8	2012	06	132.2
2001	03	30.3	2012	07	59.1
2001	04	46.1	2012	08	68.8
2001	05	34.0	2012	09	72.8
2001	06	26.8	2012	10	65.6
2001	07	40.3	2012	11	68.6
2001	08	70.0	2012	12	66.9
2001	09	110.6	2013	01	21.5
2001	10	60.5	2013	02	25.5
2001	11	26.4	2013	03	7.4
2001	12	35.2	2013	04	38.3
2002	01	55.0	2013	05	47.9
2002	02	47.6	2013	06	61.3
2002	03	26.9	2013	07	54.6
2002	04	29.9	2013	08	54.1
2002	05	72.1	2013	09	26.2
2002	06	98.8	2013	10	53.9
2002	07	80.5	2013	11	46.4
2002	08	8.2	2013	12	42.3
2002	09	16.1	2014	01	62.6
2002	10	60.5	2014	02	43.7
2002	11	53.7	2014	03	36.6
2002	12	20.9	2014	04	31.8
2003	01	17.7	2014	05	58.0
2003	02	9.1	2014	06	75.1
2003	03	6.9	2014	07	23.0
2003	04	51.4	2014	08	90.8
2003	05	68.5	2014	09	48.5

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
2003	06	66.7	2014	10	72.1
2003	07	113.5	2014	11	52.4
2003	08	55.3	2014	12	35.5
2003	09	8.9	2015	01	59.1
2003	10	45.6	2015	02	24.8
2003	11	54.5	2015	03	42.6
2003	12	52.1	2015	04	15.3
2004	01	26.4	2015	05	93.2
2004	02	27.4	2015	06	78.7
2004	03	28.5	2015	07	119.1
2004	04	20.6	2015	08	27.0
2004	05	39.7	2015	09	101.3
2004	06	56.9	2015	10	2.3
2004	07	96.0	2015	11	63.9
2004	08	75.5	2015	12	28.9
2004	09	37.2	2016	01	22.3
2004	10	78.2	2016	02	21.6
2004	11	54.1	2016	03	20.0
2004	12	24.2	2016	04	26.9
			2016	05	29.3
			2016	06	47.7
			2016	07	0.0

1 Questions

1. **How do you modify a *.q script to submit a job?** We only modified the paths to our input files and selected which BDA file to run.
2. **For which assignment do you use "local" option or "yarn" option? (you need to pick one assignmnet and show you use a different option to submit the job)** For Assignment 2, we have used `run_yarn.q` and for assignment 3, we have used `run_local.q`
3. **Given an RDD in the format ((timestamp, city), (precipitation, temperature)), what is the key? How can you reduce the data (i.e., remove timestamp from the key) to keep only the maximum temperature per city while retaining the associated precipitation value? For the second question, you should describe how to transform and operate on the data (e.g., how to write the necessary lambda functions).**Given the RDD format: ((timestamp, city), (precipitation, temperature)).

   In Spark's Key-Value Pair RDDs, the key is the first element of the main tuple.Therefore, the key is the composite tuple: (timestamp, city). The value is: (precipitation, temperature).

To achieve this, we perform a two-step transformation: an initial map to change the key, followed by a reduceByKey to find the maximum temperature.

```
city_rdd = rdd.map(lambda x:  (x[0][1], x[1]))
max_temp_rdd = city_rdd.reduceByKey(lambda a, b:  a if a[1] > b[1] else b)
```

2 Code

```
from pyspark.sql import Row, SparkSession
from pyspark.sql import functions as F

APP_NAME = "Spark SQL Demo"
spark = SparkSession.builder \
    .appName(APP_NAME) \
    .getOrCreate()

sc = spark.sparkContext # create a SparkContext object to connect to the cluster

# 1. Read the file as an RDD (assignment 1 and 4)
hdfs_as_RDD_temp = sc.textFile("hdfs:///user/x_ahmso/BDA/input/temperature-readings.csv")
hdfs_as_RDD_precip = sc.textFile("hdfs:///user/x_ahmso/BDA/input/precipitation-readings.csv")
# 2. Parse into DataFrame Rows
lines = hdfs_as_RDD_temp.map(lambda line: line.split(";")) #create an RDD of lists, where each list co
line_precip = hdfs_as_RDD_precip.map(lambda line: line.split(";"))

# assignment 5
file_as_RDD_precip = sc.textFile("file:///home/x_ahmso/BDA-lab/input_data/precipitation-readings.csv")
file_as_RDD_ostgotland = sc.textFile("file:///home/x_ahmso/BDA-lab/input_data/stations-Ostergotland.cs
lines_precip = file_as_RDD_precip.map(lambda line: line.split(";")) #create an RDD of lists, where eac
lines_ostgotland = file_as_RDD_ostgotland.map(lambda line: line.split(";")) # rdd of station names in

# -----------------------assignment 1-----------------------------------------------
# structured_RDD_temp = lines.map(lambda x: Row(year = int(x[1][0:4]), stationID = x[0], temperature =
# # 3. Create DataFrame and Filter Years
 df = spark.createDataFrame(structured_RDD_temp)
 filtered_df = df.filter((F.col('year') >= 1950) & (F.col('year') <= 2014))
 base_df = filtered_df.alias("base")

 max_temps = filtered_df.groupBy('year').agg(F.max('temperature').alias('maxValue'))
 max_df = max_temps.alias("max") #max_df is a DataFrame that contains the maximum temperature for each
 max_result = base_df.join(max_df,
                           (F.col("base.year") == F.col("max.year")) &
                           (F.col("base.temperature") == F.col("max.maxValue")),
                           'inner') \
     .select(F.col("base.year").alias("year"), F.col("base.stationID").alias("stationID"), F.col("max.
     .dropDuplicates(["year"]) \
     .orderBy(F.col("value"), descending=True)

 min_temps = filtered_df.groupBy('year').agg(F.min('temperature').alias('minValue'))
```

```
min_df = min_temps.alias("min") #min_df is a DataFrame that contains the minimum temperature for each
min_result = base_df.join(min_df,
                          (F.col("base.year") == F.col("min.year")) &
                          (F.col("base.temperature") == F.col("min.minValue")),
                          'inner') \
    .select(F.col("base.year").alias("year"), F.col("base.stationID").alias("stationID"), F.col("min.
    .dropDuplicates(["year"]) \
    .orderBy(F.col("value"), descending=True)

max_formatted = max_result.rdd.map(lambda row: f"{row.year}, {row.stationID}, {row.value}")
min_formatted = min_result.rdd.map(lambda row: f"{row.year}, {row.stationID}, {row.value}")

combined_output = max_formatted.union(min_formatted)
combined_output.saveAsTextFile("hdfs:///user/x_ahmso/BDA/output")


# ----------------------assignment 2-------------------------------------------------
parsed_data = lines.map(lambda x: Row(
    station=x[0],
    year=int(x[1][0:4]),
    month=int(x[1][5:7]),
    temperature=float(x[3])
))
df = spark.createDataFrame(parsed_data)
df_filtered = df.filter((F.col("year") >= 1950) & (F.col("year") <= 2014) & (F.col("temperature") > 10

count_all_df = df_filtered.groupBy("year", "month").count() \
                          .withColumnRenamed("count", "value") \
                          .orderBy(F.col("value").desc())

count_all_df.rdd.map(lambda row: f"{row.year}, {row.month}, {row.value}") \
                .saveAsTextFile("hdfs:///user/x_fanya/BDA/output/A2_part1")

# Group by year and month, but count only unique stations (distinct readings)
count_distinct_df = df_filtered.groupBy("year", "month") \
                               .agg(F.countDistinct("station").alias("value")) \
                               .orderBy(F.col("value").desc())

# Format output as requested: Year, month, count
count_distinct_df.rdd.map(lambda row: f"{row.year}, {row.month}, {row.value}") \
                     .saveAsTextFile("hdfs:///user/x_fanya/BDA/output/A2_part2")
# ----------------------assignment 3-------------------------------------------------
df_filtered = df.filter((F.col("year") >= 1960) & (F.col("year") <= 2014))

# Calculate average monthly temperature for each station
# Group by year, month, and station, then calculate the average temperature
avg_df = df_filtered.groupBy("year", "month", "station") \
                    .agg(F.avg("temperature").alias("avgMonthlyTemperature")) \
                    .orderBy(F.col("avgMonthlyTemperature").desc())

# Format output as requested: Year, month, station number, average monthly temperature
```

```
avg_df.rdd.map(lambda row: f"{row.year}, {row.month}, {row.station}, {row.avgMonthlyTemperature}") \
          .saveAsTextFile("file:///home/x_fanya/BDA-lab/BDA_demo/myoutput/A3")


# ----------------------------------assignment 4-------------------------------------------
structured_RDD_temp = lines.map(lambda x: Row(stationID = x[0], temperature = float(x[3])))
df_temp = spark.createDataFrame(structured_RDD_temp)
max_temp = df_temp.groupBy('stationID').agg(F.max('temperature').alias('maxValue'))

structured_RDD_precip = line_precip.map(lambda x: Row(stationID = x[0], date = x[1], precipitation = f
df_precip = spark.createDataFrame(structured_RDD_precip)
daily_precip = df_precip.groupBy('stationID', 'date').agg(F.sum('precipitation').alias('daily_precip')
max_daily_precip = daily_precip.groupBy('stationID').agg(F.max('daily_precip').alias('maxDailyPrecip')

result_temp = max_temp.filter((F.col('maxValue') >= 25.0) & (F.col('maxValue') <= 30.0))

result_precip = max_daily_precip.filter((F.col('maxDailyPrecip') >= 50.0) & (F.col('maxDailyPrecip') <

final_result = result_temp.join(result_precip, "stationID", "inner") \
    .select("stationID", "maxValue", "maxDailyPrecip") \
    .orderBy("stationID", descending=True)

final_result_formatted = final_result.rdd.map(lambda row: f"{row.stationID}, {row.maxValue}, {row.maxD
final_result_formatted.saveAsTextFile("hdfs:///user/x_ahmso/BDA/output")

# ------------------------------assignment 5-----------------------------------------
structured_RDD_precip = lines_precip.map(lambda x: Row(stationID = x[0], year = x[1][0:4], month = x[1
df_precip = spark.createDataFrame(structured_RDD_precip)
df_precip = df_precip.filter((F.col('year') >= "1993") & (F.col('year') <= "2016"))

stallions_ostgotland = lines_ostgotland.map(lambda x: Row(stationID = x[0]))
df_ostgotland = spark.createDataFrame(stallions_ostgotland)

regional_precip = df_precip.join(F.broadcast(df_ostgotland), "stationID", "inner")

monthly_total_precip = regional_precip.groupBy("stationID", "year", "month").agg(F.sum("precipitation"
avg_monthly_precip = monthly_total_precip.groupBy("year", "month").agg(F.avg("total_monthly_precip").a


final_result = avg_monthly_precip.orderBy(["year", "month"], descending=[True, True])

final_result_formatted = final_result.rdd.map(lambda row: f"{row.year}, {row.month}, {row.avg_monthly_
final_result_formatted.saveAsTextFile("file:///home/x_ahmso/BDA-lab/myoutput/")


spark.stop()
```